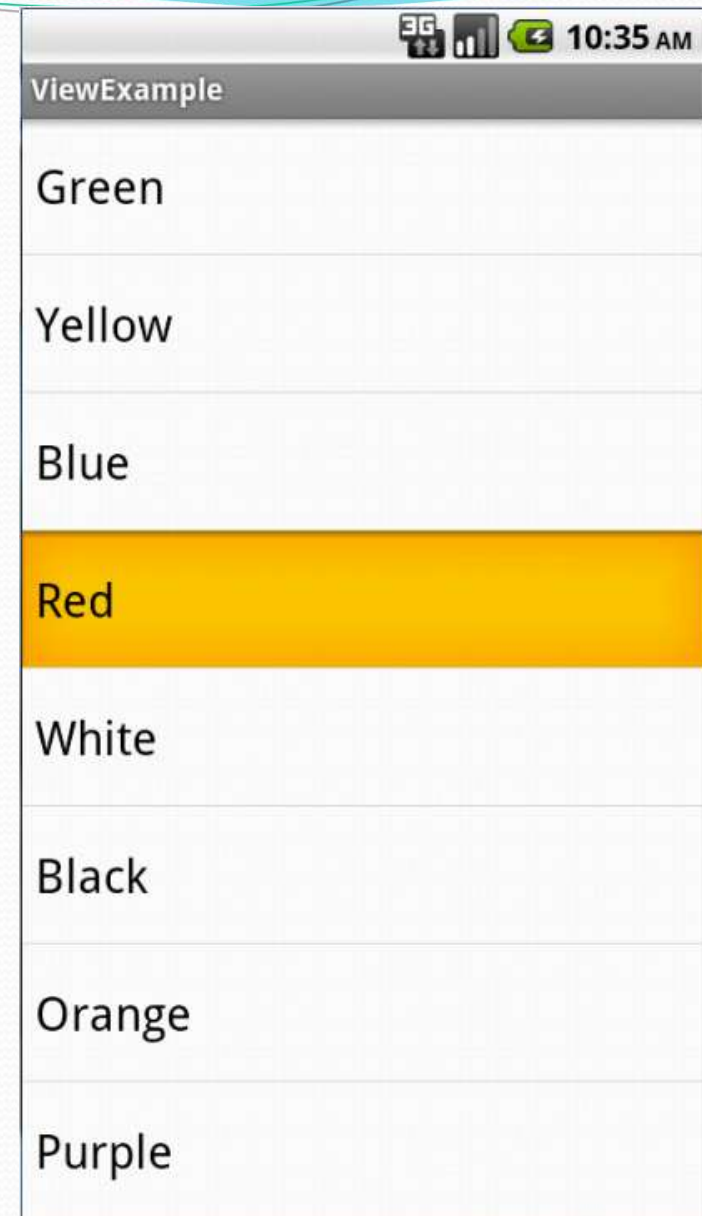


Practical 2 (References)



ListView

- A View Group that creates and manages a vertical list of Views, displaying them as rows within the list.
- The simplest List View displays the **toString** value of each object in an array, using a Text View for each item.
- Created using **ListActivity**



ListView - example

```
public class ColorActivity extends ListActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        String[] colors = {"Green", "Yellow", "Blue", "Red", "White",
                           "Black", "Orange", "Purple", "Pink", "Grey"};

        ArrayAdapter<String> adapter = new ArrayAdapter<String>(
            this, android.R.layout.simple_list_item_1, colors);

        setListAdapter(adapter);
    }
}
```

Button Click

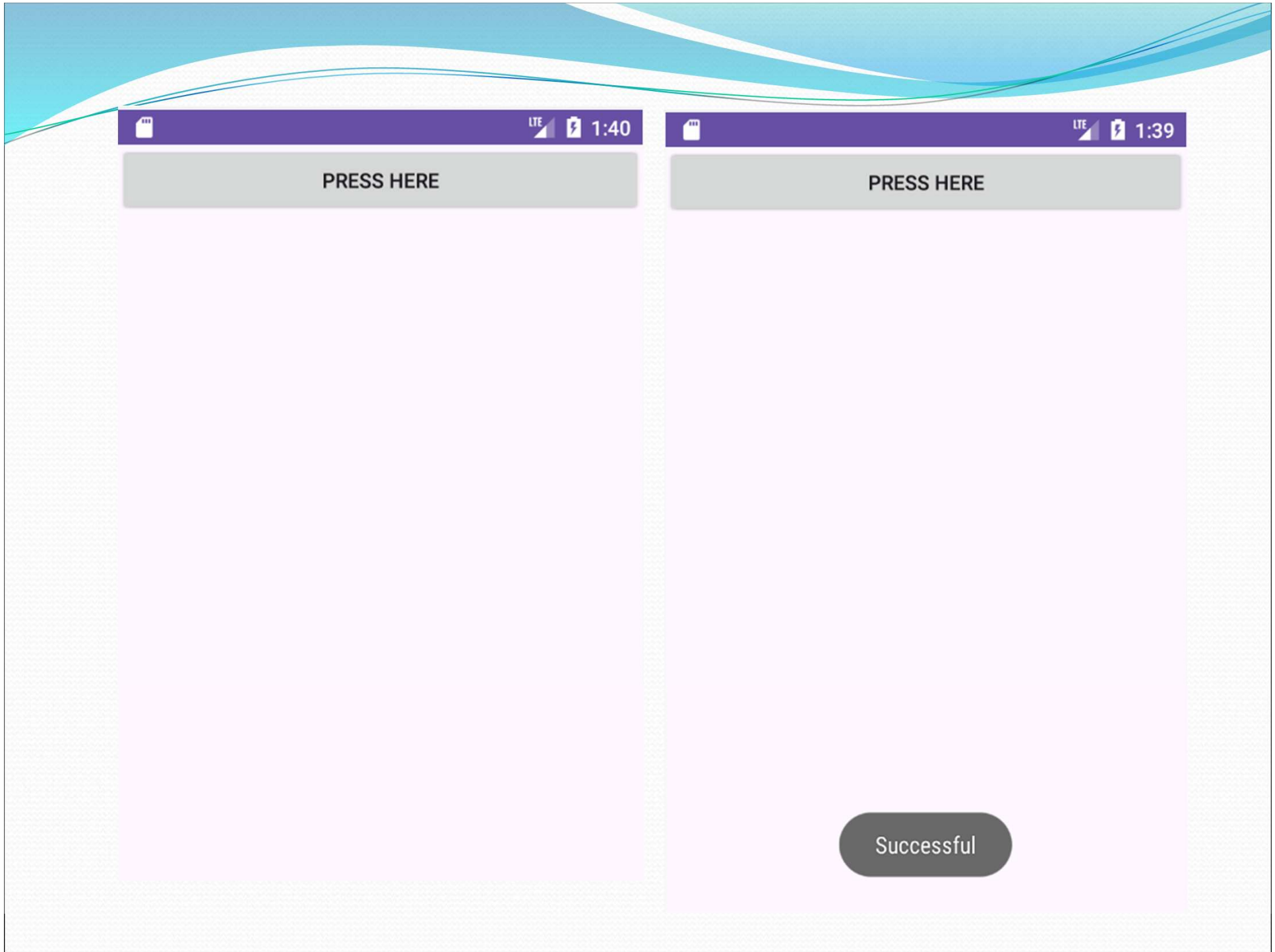
- To capture a click on the Button, the **OnClickListener** is used
- The **OnClickListener** will be set on the button and listens for on click events
- When a click is triggered, the **OnClickListener** will pick it up and asked the **onClick()** method to execute
- Usually a button is used to confirmed the selection or a checkbox group and radiobutton group
- Import: **android.view.View** and **android.view.View.OnClickListener**

Button Click - example

```
Button button = new Button(this);
button.setText("Press Here");
button.setOnClickListener(new View.OnClickListener() {

    public void onClick(View v) {
        Toast.makeText(MainActivity.this, "Successful",
            Toast.LENGTH_SHORT).show();
    }
});

LinearLayout ll = new LinearLayout(this);
ll.setOrientation(LinearLayout.VERTICAL);
ll.addView(button);
setContentView(ll);
```





Working with other Views

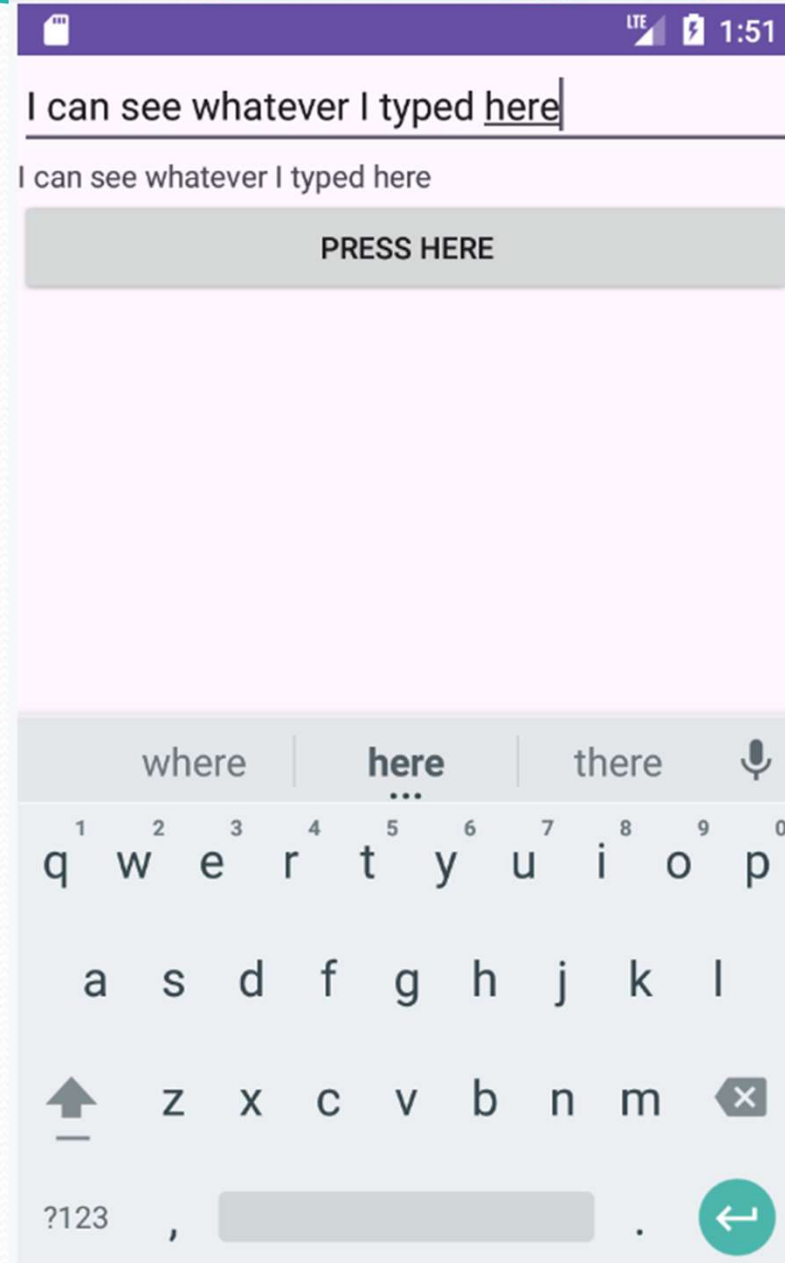
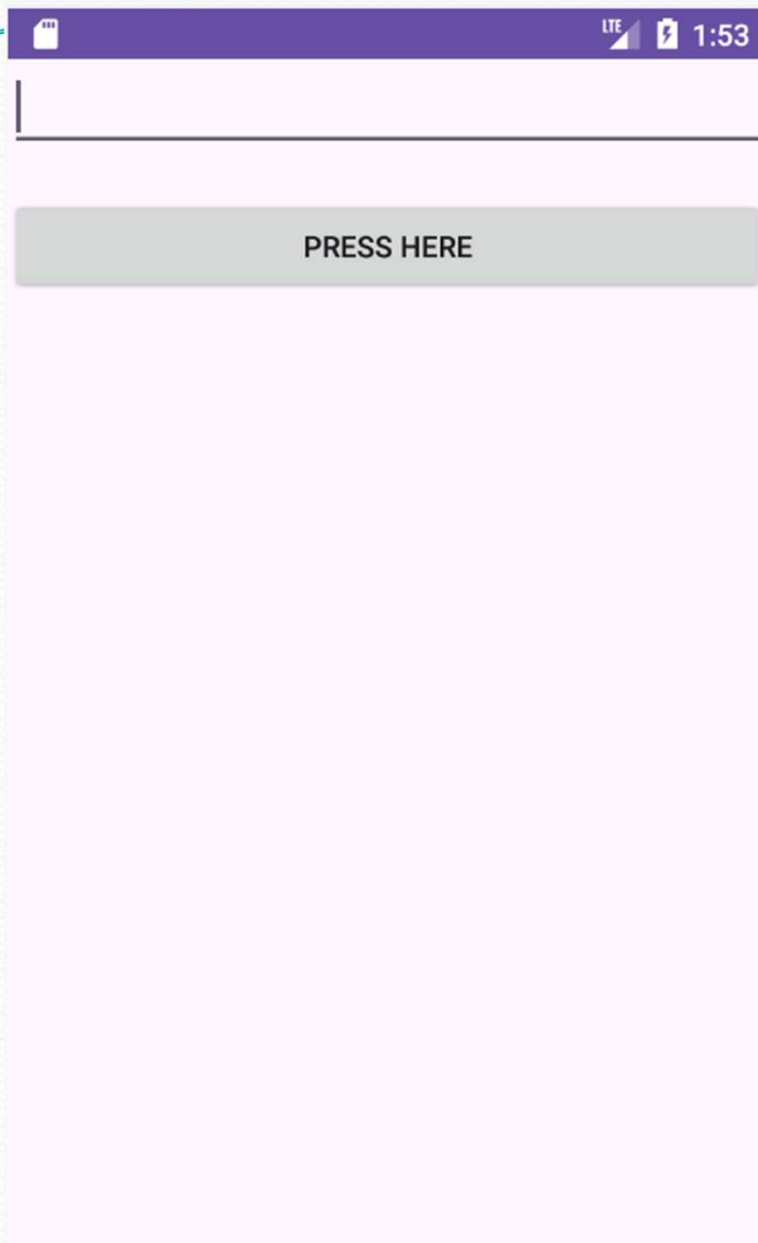
- The button click is a common method used to trigger a response
- It can be used to work with other Views that did not have the capability to respond by itself
- For example, to confirm selection of checkboxes or radiobuttons

Button Click - example 2

```
final EditText et = new EditText(this);  
final TextView tv = new TextView(this);
```

```
Button button = new Button(this);  
button.setText("Press Here");  
button.setOnClickListener(new View.OnClickListener() {  
    public void onClick(View v) {  
        tv.setText(et.getText().toString());  
    }  
});
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(et);  
ll.addView(tv);  
ll.addView(button);  
setContentView(ll);
```



RadioButton Checking Changes

- For RadioButton, other than confirming an action upon a button click, actions can be performed at every change of choice
- To listen to the changes of the choices in RadioButton, the **OnCheckedChangeListener** is used
- When the listener picks up a change, method **onCheckedChanged()** would be called
- This allows changes to happen while selecting a different choice

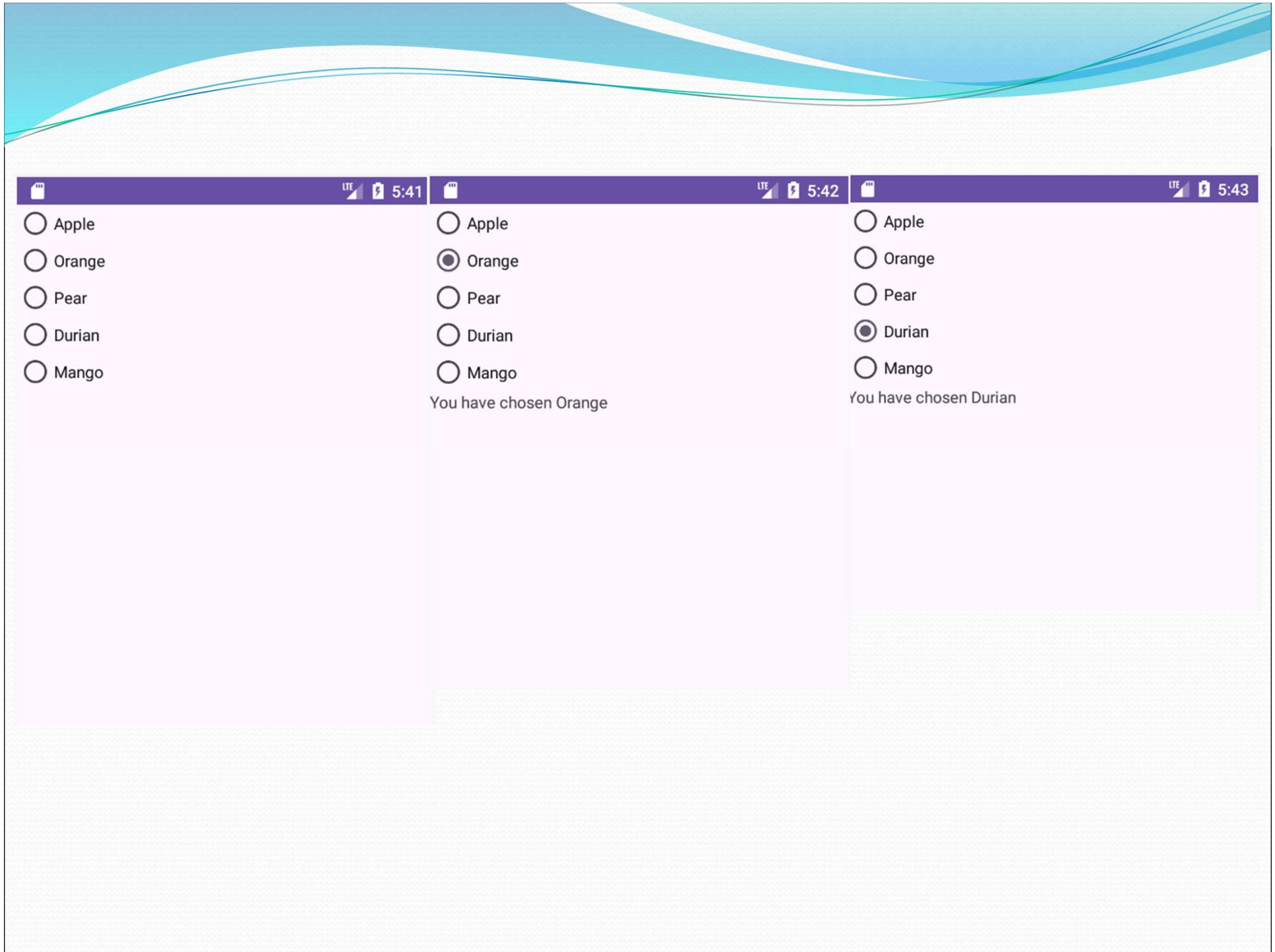
On Checked Change - example

```
TextView tv = new TextView(this);
RadioGroup rg = new RadioGroup(this);
RadioButton rb1 = new RadioButton(this);
RadioButton rb2 = new RadioButton(this);
RadioButton rb3 = new RadioButton(this);
RadioButton rb4 = new RadioButton(this);
RadioButton rb5 = new RadioButton(this);
rb1.setText("Apple");
rb2.setText("Orange");
rb3.setText("Pear");
rb4.setText("Durian");
rb5.setText("Mango");
rg.addView(rb1);
rg.addView(rb2);
rg.addView(rb3);
rg.addView(rb4);
rg.addView(rb5);
```

On Checked Change - example

```
rg.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener() {  
    public void onCheckedChanged(RadioGroup group, int checkedId) {  
        RadioButton rb =  
            (RadioButton) findViewById(rg.getCheckedRadioButtonId());  
        tv.setText("You have chosen " + rb.getText());  
    }  
});
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(rg);  
ll.addView(tv);  
setContentView(ll);
```



5:41

- ☐ Apple
- ☐ Orange
- ☐ Pear
- ☐ Durian
- ☐ Mango

5:42

- ☐ Apple
- ☒ Orange
- ☐ Pear
- ☐ Durian
- ☐ Mango

You have chosen Orange

5:43

- ☐ Apple
- ☐ Orange
- ☐ Pear
- ☒ Durian
- ☐ Mango

You have chosen Durian

Basic Intent

- Intents are used as a data-passing or navigation that works both within application or between applications
- If we have a second Activity as such:

```
public class DifferentActivity extends Activity{  
    protected void onCreate(Bundle savedInstanceState)  
    {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.different);  
    }  
}
```

Using Intent

- We can link to the new Activity (DifferentActivity) from our previous (MainActivity) by using intent
- Intent will switch the focus to the target Activity
- The target Activity will execute its content which may show a different layout

```
Intent i = new  
    Intent(MainActivity.this, DifferentActivity.class);  
startActivity(i);
```

Passing Data using Intent

```
Intent i = new  
    Intent(MainActivity.this, DifferentActivity.class);  
i.putExtra("Indicator", "value");  
startActivity(i);
```

DifferentActivity:

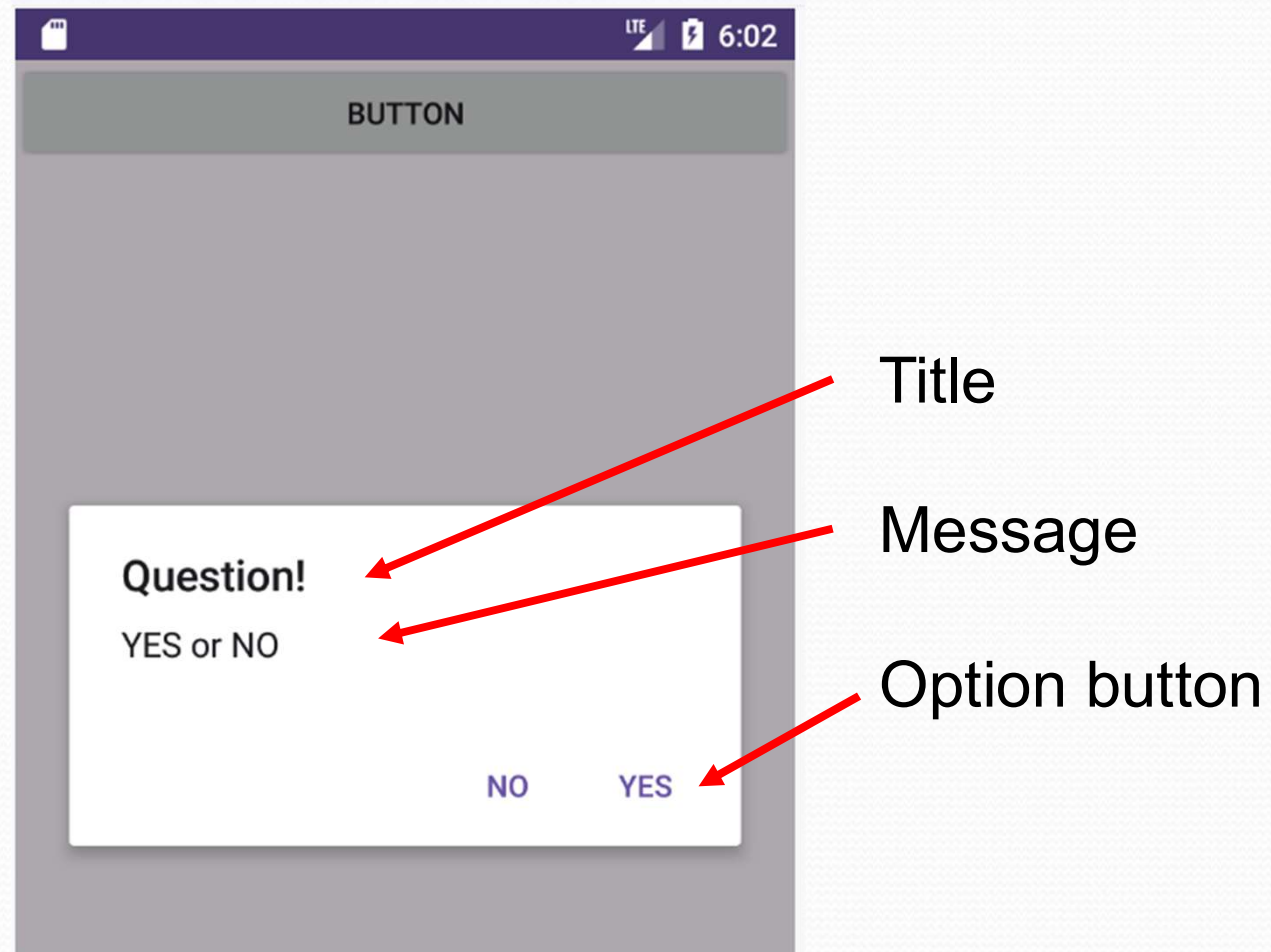
```
String string =  
    this.getIntent().getStringExtra("Indicator");
```

Toasts

- Toasts are often used to show short temporary pop-up messages

```
Toast.makeText(getApplicationContext(), "message",  
    Toast.LENGTH_LONG).show();
```

Dialog Box (AlertDialog)



Building the dialog

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Question!");
builder.setMessage("YES or NO");
builder.setCancelable(false);

builder.setPositiveButton("YES", new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int which) {
        Toast.makeText(getApplicationContext(), "YES",
            Toast.LENGTH_SHORT).show();
        finish();
    }
});

builder.setNegativeButton("NO", new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int which) {
        dialog.cancel();
    }
});
```

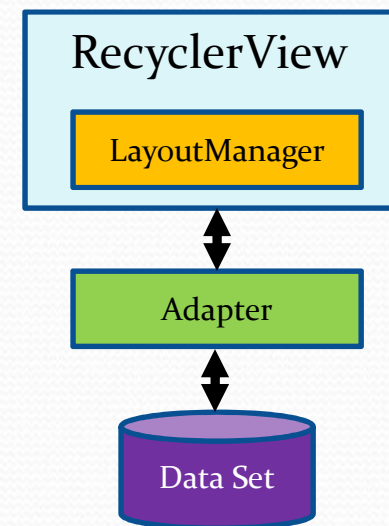
Creating the dialog

```
AlertDialog alert = builder.create();
```

```
button1.setOnClickListener(new  
    View.OnClickListener() {  
        public void onClick(View v) {  
            alert.show();  
        }  
    }) ;
```

Recycler View

- A RecyclerView in Android is a flexible and efficient container for displaying large sets of data.
 - More optimised than ListView for large data sets, with performance in mind for dynamic lists.
 - More efficient in terms of memory usage and smooth scrolling.
- Main components in a RecyclerView are:
 - RecyclerView widget, LayoutManager, Adapter, ViewHolder



Recycler View

- LayoutManager
 - LinearLayoutManager (for a vertical or horizontal list)
 - GridLayoutManager (for a grid)
- Adapter
 - For binding data to the RecyclerView
- ViewHolder
 - Container for individual views of each item in the list. It holds references to the views within a single item.

Recycler View

- ViewHolder (cont'd)
 - Simple single view design (e.g. text_row_item.xml)



Frame with margin etc

- More sophisticated multi-view design (e.g. CardView)

